

Pack 27 (API + Web) — KMS Envelope Encryption for Signing Keys + Key Rotation (real code)

RemoteDesk · Pack 27 (API + Web) — KMS Envelope Encryption for Signing Keys + Key Rotation (real code)

Codex / Claude Code handoff. Direct continuation of Pack 26. Pack 26 flagged one production blocker: the Ed25519 private key sat in a plaintext DB column. This slice wraps it with envelope encryption (a KMS-held master key encrypts a per-key data key; the data key encrypts the private key), adds a pluggable KMS provider (with a safe dev fallback that is NOT production), and builds signing-key rotation so old receipts still verify under their original key.

Scope: encrypt signing private keys at rest, decrypt only in-memory at sign time, rotate keys without breaking old signatures. **Plaintext private keys never persisted, never logged, never returned. Dev fallback is clearly marked non-production. Rotation retires, never deletes.**

Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

CREATED [NEW]	
packages/shared/src/permissions/keyRotation.ts	
apps/api/src/lib/kms/kmsProvider.ts	
apps/api/src/lib/kms/localDevKms.ts	
apps/api/src/lib/kms/envelope.ts	
apps/api/src/services/admin/keyRotation.service.ts	
apps/api/prisma/migrations/0027_key_envelope/migration.sql	
apps/web/app/dashboard/receipts/SigningKeyPanel.tsx	
tests/admin/envelope.test.ts	
tests/admin/keyRotation.test.ts	
tests/admin/keyRotation.service.test.ts	
MODIFIED [MODIFY]	
packages/shared/src/permissions/index.ts	(export keyRotation)
apps/api/prisma/schema.prisma	(encrypted key columns; drop plaintext privateKey)
apps/api/src/repositories/signingKeyRepository.ts	(store/load via envelope; no plaintext)
apps/api/src/services/admin/receiptSigner.service.ts	(decrypt-at-sign through KMS)
apps/api/src/routes/deviceAdmin.routes.ts	(admin rotate route + key list)
apps/web/app/dashboard/receipts/page.tsx	(mount SigningKeyPanel)
NOT TOUCHED (intentionally)	
apps/desktop/**	(signing is server-side)
packages/shared/src/permissions/receiptSignature.ts	(Pack 26 verify contract unchanged)
apps/api/src/lib/ed25519.ts	(sign/verify primitives unchanged)

Part A — Shared: key rotation contract

FILE: packages/shared/src/permissions/keyRotation.ts

```
import type { SigningPublicKey } from './receiptSignature';

/**
 * Key rotation model. Rotation creates a NEW active key and RETIRES the previous
 * one (never deletes), so signatures made under the old keyId still verify via
 * publicKeyById. Pure helpers for selecting the active key + describing state.
 */
export interface KeyRotationState {
  activeKeyId: string | null;
  retiredKeyIds: string[];
  lastRotatedAt: string | null;
}

/** Build rotation state from a key list (newest active wins). */
export function deriveRotationState(keys: SigningPublicKey[]): KeyRotationState {
  const active = keys.find((k) => k.retiredAt === null) ?? null;
  const retired = keys.filter((k) => k.retiredAt !== null);
  const lastRotated = retired.map((k) => k.retiredAt as string).sort().pop() ?? null;
  return {
    activeKeyId: active?.keyId ?? null,
    retiredKeyIds: retired.map((k) => k.keyId),
    lastRotatedAt: lastRotated,
  };
}

/** Is a given key id still usable for NEW signatures? Only the active key is. */
export function isActiveKey(state: KeyRotationState, keyId: string): boolean {
  return state.activeKeyId === keyId;
}

/** A retired key can still VERIFY old signatures, just not sign new ones. */
export function canVerifyWithKey(keys: SigningPublicKey[], keyId: string): boolean {
  return keys.some((k) => k.keyId === keyId);
}
```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```
// ...existing exports through Pack 26...
export * from './keyRotation'; // ADD
```

Part B — API: KMS provider interface + dev fallback

FILE: apps/api/src/lib/kms/kmsProvider.ts

```
/**
 * Pluggable KMS interface. A provider wraps/unwraps a per-key DATA KEY using a
 * KMS-held MASTER KEY. The data key (plaintext) only ever exists in memory; at
 * rest we keep just the wrapped (encrypted) data key. Implement this against AWS
 * KMS / GCP KMS / Vault Transit in production.
 */
export interface KmsProvider {
  generateDataKey(): Promise<{ plaintext: Buffer; wrapped: string }>;
  unwrap(wrapped: string): Promise<Buffer>;
}
```

```
readonly masterKeyRef: string;
}
```

FILE: apps/api/src/lib/kms/localDevKms.ts

```
import { createCipheriv, createDecipheriv, randomBytes, scriptSync } from 'node:crypto';
import type { KmsProvider } from './kmsProvider';

/**
 * LOCAL DEV KMS — NOT FOR PRODUCTION. Derives a master key from an env secret and
 * wraps data keys with AES-256-GCM. Use a real KMS in production. If
 * REMOTEDESK_KMS_DEV_SECRET is unset, it refuses to construct (fail closed)
 * rather than using a hardcoded key.
 */
export class LocalDevKms implements KmsProvider {
  readonly masterKeyRef = 'local-dev-kms';
  private readonly master: Buffer;

  constructor(secret = process.env.REMOTEDESK_KMS_DEV_SECRET) {
    if (!secret) {
      throw new Error('LocalDevKms requires REMOTEDESK_KMS_DEV_SECRET (dev only). Use a real KMS in production.');
```

FILE: apps/api/src/lib/kms/envelope.ts

```
import { createCipheriv, createDecipheriv, randomBytes } from 'node:crypto';
import type { KmsProvider } from '../kmsProvider';

/**
 * Envelope encryption for signing private keys:
 * - KMS generates a data key (wrapped at rest, plaintext in memory)
 * - we AES-256-GCM encrypt the private key with the plaintext data key
 * - we persist { wrappedDataKey, iv, tag, ciphertext } — NEVER the plaintext key
 */
export interface EncryptedKeyMaterial {
  wrappedDataKey: string;
  iv: string;
  tag: string;
  ciphertext: string;
}

export async function encryptPrivateKey(kms: KmsProvider, privateKeyB64Url: string): Promise<EncryptedKeyMaterial> {
  const { plaintext: dataKey, wrapped } = await kms.generateDataKey();
  const iv = randomBytes(12);
  const cipher = createCipheriv('aes-256-gcm', dataKey, iv);
  const ct = Buffer.concat([cipher.update(Buffer.from(privateKeyB64Url, 'utf8')), cipher.final()]);
  const tag = cipher.getAuthTag();
  dataKey.fill(0); // zero the plaintext data key ASAP
  return { wrappedDataKey: wrapped, iv: iv.toString('base64'), tag: tag.toString('base64'), ciphertext:
ct.toString('base64') };
}

export async function decryptPrivateKey(kms: KmsProvider, m: EncryptedKeyMaterial): Promise<string> {
  const dataKey = await kms.unwrap(m.wrappedDataKey);
  const decipher = createDecipheriv('aes-256-gcm', dataKey, Buffer.from(m.iv, 'base64'));
  decipher.setAuthTag(Buffer.from(m.tag, 'base64'));
  const pt = Buffer.concat([decipher.update(Buffer.from(m.ciphertext, 'base64')), decipher.final()]);
  dataKey.fill(0); // zero the plaintext data key ASAP
  return pt.toString('utf8');
}
```

Part C — API: Prisma schema + migration (encrypted columns)

FILE: apps/api/prisma/schema.prisma [MODIFY]

```
// ---- MODIFY SigningKey: replace plaintext privateKey with envelope columns ----
// model SigningKey {
//   id          String    @id @default(cuid())
//   keyId       String    @unique
//   orgId       String?
//   alg         String    @default("ed25519")
//   publicKey   String
//   // REMOVED: privateKey String
//   encWrappedDataKey String
//   encIv       String
//   encTag      String
//   encCiphertext String
//   kmsRef      String
```

```
// createdAt      DateTime @default(now())
// retiredAt      DateTime?
// @@index([orgId])
// }
```

FILE: apps/api/prisma/migrations/0027_key_envelope/migration.sql

```
-- Add envelope columns
ALTER TABLE "SigningKey" ADD COLUMN IF NOT EXISTS "encWrappedDataKey" TEXT;
ALTER TABLE "SigningKey" ADD COLUMN IF NOT EXISTS "encIv" TEXT;
ALTER TABLE "SigningKey" ADD COLUMN IF NOT EXISTS "encTag" TEXT;
ALTER TABLE "SigningKey" ADD COLUMN IF NOT EXISTS "encCiphertext" TEXT;
ALTER TABLE "SigningKey" ADD COLUMN IF NOT EXISTS "kmsRef" TEXT;

-- NOTE: migrate any existing dev rows out-of-band (re-generate keys), then:
ALTER TABLE "SigningKey" DROP COLUMN IF EXISTS "privateKey";
```

Migration note (REVIEW_REQUIRED): if Pack 26 already created plaintext-key rows in a real environment, re-issue those keys before dropping privateKey. The drop is intentional — no plaintext key column survives.

Part D — API: key repo + signer via envelope

FILE: apps/api/src/repositories/signingKeyRepository.ts [MODIFY]

```
// ...existing imports...
import { resolveKmsProvider } from '../lib/kms/localDevKms';
import { encryptPrivateKey, decryptPrivateKey, type EncryptedKeyMaterial } from '../lib/kms/envelope';
import { generateEd25519 } from '../lib/ed25519';

const kms = resolveKmsProvider();

// ---- MODIFY ensureKey: encrypt the private key via envelope before persisting ----
// async ensureKey(orgId) {
//   const active = await prisma.signingKey.findFirst({ where: { orgId, retiredAt: null }, orderBy: { createdAt: 'desc' }
// });
//   if (active) return { keyId: active.keyId }; // no private key returned anymore
//   const { publicKey, privateKey } = generateEd25519();
//   const enc = await encryptPrivateKey(kms, privateKey);
//   const keyId = `k_${Date.now().toString(36)}`;
//   await prisma.signingKey.create({ data: {
//     keyId, orgId, alg: 'ed25519', publicKey,
//     encWrappedDataKey: enc.wrappedDataKey, encIv: enc.iv, encTag: enc.tag,
//     encCiphertext: enc.ciphertext, kmsRef: kms.masterKeyRef,
//   }});
//   return { keyId };
// }

// ---- ADD: decrypt-at-sign accessor. Plaintext for IN-MEMORY use only. ----
// async loadPrivateForSigning(keyId: string): Promise<{ keyId: string; privateKey: string } | null> {
//   const k = await prisma.signingKey.findUnique({ where: { keyId },
//     select: { keyId: true, encWrappedDataKey: true, encIv: true, encTag: true, encCiphertext: true } });
//   if (!k) return null;
//   const material: EncryptedKeyMaterial = { wrappedDataKey: k.encWrappedDataKey!, iv: k.encIv!, tag: k.encTag!,
//     ciphertext: k.encCiphertext! };
//   return { keyId: k.keyId, privateKey: await decryptPrivateKey(kms, material) };
// }
```

```
// ---- ADD: rotate — retire current active key, create a fresh one ----
// async rotate(orgId: string | null): Promise<string> {
//   await prisma.signingKey.updateMany({ where: { orgId, retiredAt: null }, data: { retiredAt: new Date() } });
//   const created = await this.ensureKey(orgId);
//   return created.keyId;
// }

// ---- ADD: list keys (public material only) ----
// async listPublic(orgId: string | null): Promise<SigningPublicKey[]> {
//   const rows = await prisma.signingKey.findMany({ where: { orgId }, orderBy: { createdAt: 'desc' },
//     select: { keyId: true, publicKey: true, alg: true, createdAt: true, retiredAt: true } });
//   return rows.map((k) => ({ keyId: k.keyId, publicKey: k.publicKey, alg: 'ed25519', createdAt:
//     k.createdAt.toISOString(), retiredAt: k.retiredAt ? k.retiredAt.toISOString() : null }));
// }
//
// REMOVE the old signingPrivate() that selected a plaintext privateKey column.
```

FILE: apps/api/src/services/admin/receiptSigner.service.ts [MODIFY]

```
// ...existing Pack 26 imports...

// ---- MODIFY sign(): ensure an active key, then load+decrypt it in memory ----
// async sign(orgId, receipt) {
//   const { keyId } = await signingKeyRepository.ensureKey(orgId);
//   const loaded = await signingKeyRepository.loadPrivateForSigning(keyId);
//   if (!loaded) throw new AppError(ErrorCodes.SIGNING_KEY_UNAVAILABLE);
//   const message = signingPreimage(orgId, receipt);
//   const signature = signEd25519(loaded.privateKey, message);
//   // loaded.privateKey goes out of scope here; do NOT store or log it.
//   return { keyId, signature, alg: 'ed25519', signedAt: new Date().toISOString() };
// }
//
// publicKey() + verify() unchanged (verify still loads public key by signature.keyId).
```

Add error code (REVIEW_REQUIRED — patch errorCodes.ts): `SIGNING_KEY_UNAVAILABLE: 'SIGNING_KEY_UNAVAILABLE'`

Part E — API: rotation service + routes

FILE: apps/api/src/services/admin/keyRotation.service.ts

```
import { deriveRotationState, type KeyRotationState, type SigningPublicKey } from '@remotedesk/shared';
import { signingKeyRepository } from '../../repositories/signingKeyRepository';
import { deviceAdminAudit } from '../../lib/deviceAdminAudit'; // Pack 25

/**
 * Signing-key rotation. Retires the active key + mints a new one, audited via the
 * Pack 25 trail. Old keys remain for verifying historical receipts.
 */
export const keyRotationService = {
  async state(orgId: string): Promise<KeyRotationState> {
    const keys: SigningPublicKey[] = await (signingKeyRepository as any).listPublic(orgId);
    return deriveRotationState(keys);
  },

  async rotate(orgId: string, actorUserId: string | null): Promise<string> {
```

```

const newKeyId = await (signingKeyRepository as any).rotate(orgId);
await deviceAdminAudit.record({
  orgId, actorUserId, deviceId: 'signing-key',
  action: 'device.trust.changed', // or a dedicated 'signing.key.rotated' action
  metadata: { event: 'signing_key_rotated', newKeyId },
});
return newKeyId;
},
};

```

FILE: apps/api/src/routes/deviceAdmin.routes.ts [MODIFY]

```

// ...existing imports...
import { keyRotationService } from '../services/admin/keyRotation.service';
import { requireDeviceAdmin } from '../middleware/requireDeviceAdmin.middleware'; // Pack 24

// ---- ADD: rotation state (admin) + rotate action (admin) ----
// .get('/receipts/signing-key/state', requireAuth, orgContext, requireDeviceAdmin, h(async (req, res) => {
//   res.json(await keyRotationService.state((req as OrgRequest).orgId));
// })))
// .post('/receipts/signing-key/rotate', requireAuth, orgContext, requireDeviceAdmin, h(async (req, res) => {
//   const r = req as OrgRequest & AuthedRequest;
//   res.status(201).json({ keyId: await keyRotationService.rotate(r.orgId, r.userId) });
// })))
//
// Pack 26 GET /receipts/signing-key (public key) stays as-is and is the only route
// that returns key material — public only.

```

Part F — Web: signing key panel

FILE: apps/web/app/dashboard/receipts/SigningKeyPanel.tsx

```

'use client';

import React, { useEffect, useState } from 'react';
import type { KeyRotationState, SigningPublicKey } from '@remotedesk/shared';
import { api } from '../../../src/api/apiClient';

/**
 * Admin panel: active signing public key + rotation state + a Rotate button.
 * Never shows private material (the API can't return it).
 */
export function SigningKeyPanel({ canManage }: { canManage: boolean }) {
  const [pub, setPub] = useState<SigningPublicKey | { keyId: null } | null>(null);
  const [state, setState] = useState<KeyRotationState | null>(null);
  const [busy, setBusy] = useState(false);

  const load = async () => {
    setPub(await api.get('/devices/receipts/signing-key'));
    if (canManage) setState(await api.get<KeyRotationState>('/devices/receipts/signing-key/state'));
  };
  useEffect(() => { void load(); }, []); // eslint-disable-line react-hooks/exhaustive-deps

  const rotate = async () => {
    setBusy(true);

```

```

    try { await api.post('/devices/receipts/signing-key/rotate', {}); await load(); }
    finally { setBusy(false); }
  };

  return (
    <section className="signing-key-panel">
      <h2>Receipt signing key</h2>
      <p className="mono">
        Active key: {pub && 'keyId' in pub && pub.keyId ? pub.keyId : 'none yet (sign a receipt to create one)'}
      </p>
      {state && (
        <p className="muted">
          Retired keys: {state.retiredKeyIds.length}
          {state.lastRotatedAt ? ` · last rotated ${new Date(state.lastRotatedAt).toLocaleString()}` : ''}
        </p>
      )}
      {canManage && (
        <button type="button" disabled={busy} onClick={rotate}>
          {busy ? 'Rotating...' : 'Rotate signing key'}
        </button>
      )}
      <p className="muted">Old keys are retired, not deleted, so past receipts stay verifiable.</p>
    </section>
  );
}

```

FILE: apps/web/app/dashboard/receipts/page.tsx [MODIFY]

```

// ...existing imports...
import { SigningKeyPanel } from './SigningKeyPanel';
// import { useDeviceAdminPermissions } from '../devices/useDeviceAdminPermissions'; // Pack 24

// ---- ADD near the top of the receipts page (admin-aware) ----
// const { canMutate } = useDeviceAdminPermissions(role);
// <SigningKeyPanel canManage={canMutate} />

```

Part G — Tests

FILE: tests/admin/envelope.test.ts

```

import { describe, it, expect } from 'vitest';
import { LocalDevKms } from '../../apps/api/src/lib/kms/localDevKms';
import { encryptPrivateKey, decryptPrivateKey } from '../../apps/api/src/lib/kms/envelope';

describe('envelope encryption', () => {
  it('round-trips a private key through encrypt/decrypt', async () => {
    const kms = new LocalDevKms('test-secret');
    const priv = 'abc123_base64url_private_key';
    const enc = await encryptPrivateKey(kms, priv);
    expect(enc.ciphertext).not.toContain(priv); // not plaintext at rest
    expect(await decryptPrivateKey(kms, enc)).toBe(priv);
  });

  it('wrapped material does not reveal the private key', async () => {
    const kms = new LocalDevKms('test-secret');

```

```

    const enc = await encryptPrivateKey(kms, 'super-secret-key');
    expect(JSON.stringify(enc)).not.toContain('super-secret-key');
  });

  it('a different master cannot unwrap', async () => {
    const a = new LocalDevKms('secret-A');
    const b = new LocalDevKms('secret-B');
    const enc = await encryptPrivateKey(a, 'k');
    await expect(decryptPrivateKey(b, enc)).rejects.toThrow();
  });

  it('LocalDevKms refuses to construct without a secret (fail closed)', () => {
    expect(() => new LocalDevKms(undefined)).toThrow();
  });
});

```

FILE: tests/admin/keyRotation.test.ts

```

import { describe, it, expect } from 'vitest';
import { deriveRotationState, isActiveKey, canVerifyWithKey, type SigningPublicKey } from '@remotedesk/shared';

function key(keyId: string, retiredAt: string | null): SigningPublicKey {
  return { keyId, publicKey: `pub-${keyId}`, alg: 'ed25519', createdAt: '2026-06-15T10:00:00.000Z', retiredAt };
}

describe('keyRotation (pure)', () => {
  it('picks the non-retired key as active', () => {
    const s = deriveRotationState([key('k2', null), key('k1', '2026-06-15T09:00:00.000Z')]);
    expect(s.activeKeyId).toBe('k2');
    expect(s.retiredKeyIds).toEqual(['k1']);
  });

  it('reports lastRotatedAt from the newest retirement', () => {
    const s = deriveRotationState([key('k3', null), key('k2', '2026-06-15T09:00:00.000Z'), key('k1', '2026-06-14T09:00:00.000Z')]);
    expect(s.lastRotatedAt).toBe('2026-06-15T09:00:00.000Z');
  });

  it('isActiveKey only true for the active key', () => {
    const s = deriveRotationState([key('k2', null), key('k1', '2026-06-15T09:00:00.000Z')]);
    expect(isActiveKey(s, 'k2')).toBe(true);
    expect(isActiveKey(s, 'k1')).toBe(false);
  });

  it('retired keys can still verify (present in the list)', () => {
    const keys = [key('k2', null), key('k1', '2026-06-15T09:00:00.000Z')];
    expect(canVerifyWithKey(keys, 'k1')).toBe(true);
    expect(canVerifyWithKey(keys, 'k9')).toBe(false);
  });

  it('handles no keys yet', () => {
    const s = deriveRotationState([]);
    expect(s.activeKeyId).toBeNull();
    expect(s.lastRotatedAt).toBeNull();
  });
});

```

FILE: tests/admin/keyRotation.service.test.ts

```
import { describe, it, expect, vi, beforeEach } from 'vitest';

const listPublic = vi.fn();
const rotate = vi.fn();
const auditRecord = vi.fn().mockResolvedValue(undefined);
vi.mock('../..../apps/api/src/repositories/signingKeyRepository', () => ({
  signingKeyRepository: { listPublic: (...a: unknown[]) => listPublic(...a), rotate: (...a: unknown[]) => rotate(...a) },
}));
vi.mock('../..../apps/api/src/lib/deviceAdminAudit', () => ({
  deviceAdminAudit: { record: (...a: unknown[]) => auditRecord(...a) },
}));

import { keyRotationService } from '../..../apps/api/src/services/admin/keyRotation.service';

describe('keyRotationService', () => {
  beforeEach(() => { listPublic.mockReset(); rotate.mockReset(); auditRecord.mockClear(); });

  it('derives state from the key list', async () => {
    listPublic.mockResolvedValue([
      { keyId: 'k2', publicKey: 'p', alg: 'ed25519', createdAt: 'now', retiredAt: null },
      { keyId: 'k1', publicKey: 'p', alg: 'ed25519', createdAt: 'old', retiredAt: '2026-06-15T09:00:00.000Z' },
    ]);
    const s = await keyRotationService.state('org');
    expect(s.activeKeyId).toBe('k2');
    expect(s.retiredKeyIds).toEqual(['k1']);
  });

  it('rotates + audits', async () => {
    rotate.mockResolvedValue('k3');
    const newId = await keyRotationService.rotate('org', 'u1');
    expect(newId).toBe('k3');
    expect(rotate).toHaveBeenCalled('org');
    expect(auditRecord).toHaveBeenCalledTimes(1);
    expect(auditRecord.mock.calls[0][0].metadata.newKeyId).toBe('k3');
  });
});
```

Part H — Reports

FILE: [generated-pack27-kms-envelope-rotation-merge-summary.md](#)

```
# Pack 27 — KMS Envelope Encryption + Key Rotation • Merge Summary

## What this slice does
Closes the Pack 26 production blocker (plaintext private key at rest):

1. Envelope encryption: a KMS master key wraps a per-key data key; the data key
   AES-256-GCM-encrypts the Ed25519 private key. Only { wrappedDataKey, iv, tag,
   ciphertext } is persisted — never the plaintext key.

2. Pluggable KmsProvider with a LocalDevKms fallback (NOT production; requires an
   env secret, fail-closed without it). Swap to AWS/GCP/Vault in resolveKmsProvider.

3. Key rotation: retire the active key + mint a new one; old keys remain for
   verifying historical receipts. Audited via Pack 25.
```

```

## Files created (10) / modified (6)
See manifest. New: shared keyRotation; api kms/{provider, localDevKms, envelope}; keyRotation.service; migration; web
SigningKeyPanel; 3 tests.
Modified: shared index; schema (envelope cols + DROP privateKey); signingKeyRepository; receiptSigner.service;
deviceAdmin.routes; receipts page.

## SECURITY
- Plaintext private key + plaintext data key exist ONLY in memory at sign time; both zeroed after use.
- DB stores only encrypted material + KMS-wrapped data key; no plaintext key column survives (dropped).
- LocalDevKms is dev-only + fail-closed (no hardcoded master); production must wire a real KMS.

## Remaining manual wiring risks
1. Implement a production KmsProvider (AWS/GCP/Vault) and select it in resolveKmsProvider.
2. Re-issue any Pack 26 dev keys before dropping the plaintext privateKey column.
3. signingKeyRepository: remove the old plaintext signingPrivate(); use loadPrivateForSigning.
4. Add SIGNING_KEY_UNAVAILABLE error code (+ optional signing.key.rotated audit action).

## Verify
- REMOTEDesk_KMS_DEV_SECRET=... pnpm -r test
- Manual: sign a receipt -> DB row has encCiphertext, no privateKey; rotate -> new active keyId, old receipts still
verify.

```

FILE: [generated-pack27-kms-envelope-rotation-code-review.md](#)

```

# Pack 27 — Code Review Notes

## Strengths
- Standard envelope pattern: KMS wraps a data key; data key encrypts the private key; only ciphertext at rest.
- Plaintext key + data key are memory-only and zeroed right after use.
- KMS is an interface; LocalDevKms is clearly non-production and fails closed without a secret.
- Rotation retires (never deletes) keys, so historical signatures keep verifying via keyId lookup.
- Pure rotation logic + envelope round-trip + service all unit-tested.

## Reviewer checklist
- [ ] resolveKmsProvider points at a real KMS in production (not LocalDevKms).
- [ ] No code path returns/logs decrypted plaintext or the data key.
- [ ] Plaintext privateKey column dropped; loadPrivateForSigning is the only decrypt path.
- [ ] Rotate retires exactly the active key, mints one new active key.
- [ ] verify route still resolves public key by signature.keyId (old keys present).

## Safety
- No plaintext private key at rest, in logs, or in any response.
- Dev KMS cannot run without an explicit secret (no hardcoded fallback).
- Rotation preserves verifiability of past receipts.

```

FILE: generated-pack27-kms-envelope-rotation-manifest.json

```

{
  "pack": "pack27-kms-envelope-rotation",
  "title": "KMS Envelope Encryption for Signing Keys + Key Rotation",
  "actualFileCount": 16,
  "filesCreated": [
    "packages/shared/src/permissions/keyRotation.ts",
    "apps/api/src/lib/kms/kmsProvider.ts",
    "apps/api/src/lib/kms/localDevKms.ts",

```

```

    "apps/api/src/lib/kms/envelope.ts",
    "apps/api/src/services/admin/keyRotation.service.ts",
    "apps/api/prisma/migrations/0027_key_envelope/migration.sql",
    "apps/web/app/dashboard/receipts/SigningKeyPanel.tsx",
    "tests/admin/envelope.test.ts",
    "tests/admin/keyRotation.test.ts",
    "tests/admin/keyRotation.service.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "apps/api/prisma/schema.prisma",
    "apps/api/src/repositories/signingKeyRepository.ts",
    "apps/api/src/services/admin/receiptSigner.service.ts",
    "apps/api/src/routes/deviceAdmin.routes.ts",
    "apps/web/app/dashboard/receipts/page.tsx"
  ],
  "filesNotTouched": [
    "apps/desktop/**",
    "packages/shared/src/permissions/receiptSignature.ts",
    "apps/api/src/lib/ed25519.ts"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["@remotedesk/shared", "@prisma/client", "node:crypto", "express", "next", "react", "vitest"],
    "assumedFromProject": [
      "Pack 26 SigningKey model + receiptSigner.service",
      "Pack 25 deviceAdminAudit",
      "Pack 24 requireDeviceAdmin RBAC",
      "a production KMS to implement KmsProvider against"
    ]
  },
  "knownLimitations": [
    "LocalDevKms is NOT production — derives a master from an env secret; replace with AWS/GCP/Vault.",
    "Data-key-per-signing-key (not per-receipt); fine for Ed25519 key protection.",
    "Rotation is manual (admin button); scheduled/auto-rotation is a follow-up.",
    "Re-encryption of existing rows to a new master not included; rotation mints fresh keys instead."
  ],
  "remainingManualWiringRisks": [
    "Implement + select a production KmsProvider.",
    "Drop plaintext privateKey column after re-issuing dev keys.",
    "Wire loadPrivateForSigning + remove old plaintext signingPrivate()."
  ],
  "estimatedCompletionAfterCarefulMerge": "84-89%"
}

```

Pack 27 done: 16 files (10 created, 6 modified). Closes the Pack 26 production blocker. Signing private keys are now protected with standard envelope encryption: a KMS master key wraps a per-key data key, the data key AES-256-GCM-encrypts the Ed25519 private key, and only the encrypted material + wrapped data key are persisted. The plaintext privateKey column is dropped entirely.

The clean bit: the plaintext private key and the plaintext data key exist only in memory at sign time and are zeroed the instant they're used, so neither ever lands at rest, in a log, or in a response. The KMS is a clean interface; the LocalDevKms fallback is explicitly non-production and fails closed if no secret is set, so wiring

AWS/GCP/Vault is a one-function swap. Rotation retires keys instead of deleting them, so every historical receipt still verifies under its original keyId.

The receipt trust chain is now production-shaped end to end: consistent (22) -> verified (24) -> attributable (25) -> signed (26) -> key-protected + rotatable (27). Two things before prod: implement a real KmsProvider, and re-issue any Pack 26 dev keys before the migration drops the plaintext column. Next slice = scheduled/auto key rotation, or a desktop-side signature verify so the host confirms its own receipts offline. Bolo.